

MarketNest

Multi-Vendor E-Commerce Platform

Architecture, Stack & Planning Document — v1.0

Prepared: June 2026 · Confidential

Next.js 14	NestJS	Supabase	PostgreSQL	pgvector	Redis	Vercel
------------	--------	----------	------------	----------	-------	--------

This document is the single source of truth for the MarketNest platform. It covers the chosen technology stack and rationale, full system architecture, database design, security model, scalability strategy, CI/CD pipeline, AI roadmap, and the development phasing plan. Every decision recorded here has been made with 1 million+ concurrent users as the baseline requirement.

Table of Contents

1		
.	Executive Summary	3
2		
.	System Overview — Three Portals	3
3		
.	Technology Stack Decision	4
4		
.	Full Architecture Breakdown	5
5		
.	Database Design	6
6		
.	Authentication & Security Model	7
7		
.	Scalability & Performance Strategy	8
8		
.	Payment Integration	9
9		
.	AI / RAG Roadmap	9
10		
.	CI/CD Pipeline & DevOps	10
11		
.	Development Phases	10
12		
.	Environment Variables & Configuration	11
13		
.	Folder Structure	11

1. Executive Summary

MarketNest is a scalable multi-vendor e-commerce platform built on a modern, cloud-native stack. It supports three distinct actor roles — Super Admin, Seller, and Buyer — each with a fully isolated portal and permission scope. The architecture is designed for high availability, sub-100ms page loads globally, and horizontal scalability beyond 1 million active users without architectural changes. The AI layer using Retrieval-Augmented Generation (RAG) is built in from day one so it can be activated progressively without retrofitting.

Key decisions at a glance:

Frontend	Next.js 14 (App Router) — SSR + ISR + Edge for maximum performance
Backend	NestJS on Node.js — typed, modular microservices with strong DI
Database	Supabase PostgreSQL — managed, Row-Level Security, Realtime built in
Vector DB	Supabase pgvector — no separate vector DB needed, AI-ready from day one
Cache	Redis via Upstash — sessions, cart state, rate limiting, hot product cache
Hosting	Vercel (frontend) + Railway/Render (NestJS) — auto-scaling, zero-config
Payments	Stripe (primary) + local gateway (COD webhook) — dual mode from launch
Auth	Supabase Auth — Google OAuth + email/password + guest sessions

2. System Overview — Three Portals

Super Admin Portal

- One account, created at system bootstrap via seed script.
- Can create, suspend, and delete seller accounts.
- Views platform-wide analytics: GMV, orders, revenue, user counts.
- Manages product categories, global banners, and featured listings.
- Can override any order or payment dispute.
- Accesses full audit log of all system actions.

Seller Portal

- Account created by Super Admin; seller receives email invite.
- Manages their own product catalogue: add, edit, delete, stock control.
- Views their own order dashboard: new, processing, shipped, complete.
- Reviews customer feedback and ratings for their store.
- Sees payout history and upcoming payment schedules.
- Cannot see other sellers' data (enforced by RLS at DB level).

Buyer Storefront

- Can browse and add to cart as a guest — no account required.
- Guest cart persisted in Redis with a session token (7-day TTL).
- Prompted to sign in or create account only at checkout.
- Google OAuth one-click sign-in or email/password registration.
- On first login, guest cart is merged into the new user account.
- Supports Cash on Delivery and online payment (Stripe) at checkout.
- Real-time order tracking after purchase.

3. Technology Stack Decision

3.1 Why Next.js 14 (App Router)

Next.js 14 with the App Router is the right choice because it gives us Server Components (zero JS shipped for static UI), built-in ISR (product pages cached at CDN, revalidated on update), and Edge Middleware (auth checks run at the CDN edge, not the origin). For an e-commerce site where product pages must load in under 100ms globally, this is the only sensible frontend choice. The Vercel deployment platform pairs perfectly and provides automatic image optimisation, analytics, and global CDN out of the box.

3.2 Why NestJS over direct Supabase calls

While Supabase can be called directly from Next.js Server Actions, adding NestJS as a dedicated API layer provides: (a) a single enforced entry point for rate limiting and WAF rules, (b) clean domain-driven module separation (AuthModule, ProductModule, OrderModule, PaymentModule), (c) Guards and Interceptors for RBAC that are testable in isolation, (d) the ability to add WebSocket gateways and queue workers in the same runtime, and (e) future ability to extract individual modules into separate microservices without rewriting. For a 1M+ user platform this separation is essential — the API tier can scale independently of the frontend.

3.3 Why Supabase

Supabase provides managed PostgreSQL with Row Level Security (the cornerstone of our multi-tenant data isolation), built-in Realtime subscriptions (order status updates), Auth (Google OAuth, email/password, magic links), Storage (product images with CDN), and pgvector (AI embeddings). This eliminates the need for six separate managed services. The cost advantage at launch is significant; at scale, Supabase Pro scales to millions of requests.

3.4 Full stack summary

Layer	Technology	Reason
Frontend	Next.js 14 App Router	SSR/ISR/Edge, Vercel deploy
UI Library	Tailwind CSS + shadcn/ui	Utility-first, accessible components
Backend API	NestJS (Node.js 20)	Typed, modular, scalable
ORM	Prisma	Type-safe DB client, migrations
Primary DB	Supabase PostgreSQL 15	Managed, RLS, Realtime
Vector DB	Supabase pgvector	AI search, no extra cost
Cache	Redis (Upstash)	Sessions, cart, rate limits
File Storage	Supabase Storage	CDN-backed, signed URLs
Auth	Supabase Auth	Google OAuth, guest sessions
Queue	BullMQ + Redis	Async jobs, email workers
Payments	Stripe + COD webhook	Cards + cash on delivery
Email	SendGrid / Resend	Transactional email
SMS	Twilio	Order confirmations

Monitoring	Sentry + Datadog	Errors + APM
CI/CD	GitHub Actions + Vercel	Auto-deploy on push
Hosting FE	Vercel Pro	Edge, auto-scale
Hosting API	Railway / Render	Container, auto-scale

4. Full Architecture Breakdown

4.1 Client layer

All three portals are Next.js applications. They can be separate Next.js apps (monorepo) or route groups within a single app, depending on domain strategy. Recommended: separate sub-domains — `admin.marketnest.com`, `sell.marketnest.com`, `marketnest.com`. Each portal uses Next.js App Router with React Server Components. Static product pages use ISR with a 60-second revalidation window so CDN serves cached HTML for 99% of traffic.

4.2 CDN / Edge layer

Vercel's global edge network handles static assets, ISR pages, and Edge Middleware. Cloudflare WAF sits in front of Vercel and the NestJS API to absorb DDoS, apply geo-blocking, and run bot detection. This layer absorbs 90%+ of traffic at the CDN level — the origin servers only receive dynamic, authenticated requests.

4.3 API Gateway (NestJS)

The NestJS API is the single entry point for all mutations and authenticated reads. It runs on Railway/Render behind a load balancer. Key responsibilities: JWT validation, RBAC Guard (SuperAdmin > Seller > Buyer > Guest), request rate limiting (express-rate-limit + Redis), input validation (class-validator), request logging (Winston + Datadog transport), and Swagger docs at `/api/docs`.

4.4 Service modules (NestJS)

AuthModule — handles sign-up, sign-in, Google OAuth callback, guest token issuance, cart-merge on first login. ProductModule — CRUD for product listings, category management, inventory tracking, image upload to Supabase Storage. OrderModule — cart management in Redis, checkout pipeline, order state machine (pending > confirmed > processing > shipped > delivered > cancelled), COD flow. PaymentModule — Stripe PaymentIntent creation, webhook handling, payout calculation. NotificationModule — email via SendGrid, SMS via Twilio, push via FCM, real-time via Supabase Realtime. AIModule — embedding generation, RAG queries, recommendation engine.

4.5 Message queue (BullMQ)

Long-running tasks are offloaded to BullMQ queues backed by Redis. Queues include: email-queue (send order confirmations, invoices), inventory-queue (restock alerts), embedding-queue (generate product embeddings on create/update), export-queue (CSV exports for admin). Workers run as separate Node processes on the same Railway instance, with auto-retry and DLQ.

4.6 Data layer (Supabase)

PostgreSQL is the primary store with RLS policies enforcing data isolation between sellers. Redis (Upstash) caches: user sessions (24h TTL), guest carts (7d TTL), product listings (60s TTL invalidated on update), rate-limit counters. Supabase Storage holds product images, with a Supabase CDN transform URL for on-the-fly resizing (thumbnail, card, full). pgvector stores 1536-dim embeddings for AI-powered product search and recommendations.

5. Database Design

All tables live in Supabase PostgreSQL. Row Level Security (RLS) is enabled on every table. Supabase Auth manages the users table; application tables reference auth.users(id) via foreign key.

Table	Key columns
profiles	id (FK auth.users), role (enum: superadmin seller buyer), full_name, avatar_url, phone, created_at
sellers	id, user_id (FK profiles), store_name, store_slug, description, is_active, commission_rate, created_by (FK profiles)
categories	id, name, slug, parent_id (self-ref), image_url, sort_order, is_active
products	id, seller_id (FK sellers), category_id, title, description, price, compare_price, stock_qty, sku, images (jsonb), status (enum), embedding (vector 1536), created_at, updated_at
product_variants	id, product_id, name, options (jsonb), price_delta, stock_qty, sku
carts	id, user_id (nullable FK profiles), session_id (for guests), created_at, updated_at
cart_items	id, cart_id, product_id, variant_id, quantity, unit_price, added_at
orders	id, buyer_id (FK profiles), status (enum), payment_method (cod online), subtotal, shipping_fee, total, shipping_address (jsonb), created_at, updated_at
order_items	id, order_id, product_id, seller_id, quantity, unit_price, status
payments	id, order_id, stripe_payment_intent_id, amount, currency, status, paid_at
payouts	id, seller_id, amount, status, period_start, period_end, processed_at
reviews	id, product_id, buyer_id, rating (1-5), body, created_at
audit_logs	id, actor_id, action, entity_type, entity_id, metadata (jsonb), created_at

Key RLS policies (examples):

- products — SELECT: everyone can read active products. INSERT/UPDATE/DELETE: seller_id must match auth.uid().
- orders — SELECT: buyer_id = auth.uid() OR seller has order_items with their seller_id.
- sellers — SELECT: super admin sees all; sellers see only their own row.
- audit_logs — SELECT: super admin only. INSERT: service role only (from NestJS).

6. Authentication & Security Model

Supabase Auth

Handles all identity management. JWT access tokens (15min TTL) + refresh tokens (30d). Tokens stored in HTTP-only cookies, not localStorage.

Google OAuth

Buyer clicks 'Continue with Google'. Supabase redirects to Google, receives code, creates profile row with role=buyer automatically.

Guest session

On first page load, NestJS issues a signed guest token (UUID v4) stored in a cookie. Cart is stored in Redis keyed by this token. On login, cart is merged into user account.

Seller creation

Only super admin can create seller accounts via POST /admin/sellers. An email invite is sent via SendGrid. Seller completes registration via the invite link.

RBAC guards

NestJS Guard reads the JWT role claim. Routes are decorated with @Roles('superadmin'|'seller'|'buyer'). Any mismatch returns 403 immediately before the controller runs.

Rate limiting

express-rate-limit on all routes: 100 req/min general, 10 req/min on auth endpoints, 5 req/min on checkout. Redis stores counters. Cloudflare WAF provides the outer DDoS shield.

Input validation

All DTOs validated with class-validator. SQL injection is impossible via Prisma's parameterised queries. XSS prevented by React's default escaping.

Secrets management

All secrets in environment variables (Vercel env / Railway env). Never committed to git. Rotated quarterly. Supabase service role key used only in NestJS, never exposed to the browser.

7. Scalability & Performance Strategy

Target: handle 1 million+ concurrent users without architectural changes.

CDN-first rendering

Next.js ISR means product pages, homepage, and category pages are pre-rendered and served from Vercel's global CDN. The origin API never receives these requests. This handles 90%+ of buyer traffic.

Redis caching layers

Hot product catalogue cached in Redis with 60s TTL. User sessions in Redis (not DB). Cart state in Redis. Product page view counts in Redis (async flush to DB every 5 minutes). This removes 95% of read load from PostgreSQL.

Database connection pooling

Supabase uses PgBouncer in transaction mode. NestJS connects via the pooled connection string (port 6543). Each NestJS pod uses a pool of 10 connections. At 10 pods, this is 100 DB connections — well within Supabase Pro limits.

Read replicas

Supabase Pro supports read replicas in different regions. Heavy read queries (product search, order history) route to the nearest read replica. All writes go to the primary.

Horizontal API scaling

NestJS API is stateless (state in Redis/Postgres, not in-process). Railway/Render auto-scales pods based on CPU/memory. Zero configuration needed.

Async queue offloading

Any operation > 200ms is moved to a BullMQ queue. Emails, inventory updates, embedding generation, invoice PDFs — all async. The HTTP response returns immediately and the worker completes the work in the background.

Image optimisation

Supabase Storage serves images via a CDN transform URL. Next.js Image component serves WebP with correct dimensions for every device. No full-size images ever load in the browser.

Database indexes

Critical indexes: `products(seller_id)`, `products(category_id)`, `products(status)`, `orders(buyer_id)`, `orders(status)`, `order_items(seller_id)`, `cart_items(cart_id)`. Combined these reduce query time on hot paths from $O(n)$ to $O(\log n)$.

8. Payment Integration

Two payment modes are supported at launch. Online payment uses Stripe's PaymentIntent API. The flow is: (1) buyer reaches checkout, (2) NestJS creates a PaymentIntent and returns the client_secret, (3) Next.js renders Stripe Elements with the client_secret, (4) buyer completes payment in the browser, (5) Stripe webhook fires payment_intent.succeeded to NestJS, (6) order status moves to confirmed and fulfilment begins. Cash on Delivery (COD) creates the order in status=pending_cod and triggers a seller notification. Delivery confirmation (seller marks delivered) triggers the payout calculation. Payouts to sellers are batched weekly via Stripe Connect or manual bank transfer, after deducting the platform commission rate stored on the sellers table.

Stripe webhook events handled: payment_intent.succeeded, payment_intent.payment_failed, charge.dispute.created, account.updated (for Connect).

9. AI / RAG Roadmap

Phase 1 — Semantic search (launch)

When a seller adds or updates a product, an embedding is generated using OpenAI text-embedding-3-small (1536 dims) and stored in the product's embedding column (pgvector). The buyer search bar queries pgvector using cosine similarity. This gives Google-quality search from day one — finds 'blue cotton summer dress' even if the product title says 'azure lightweight frock'.

Phase 2 — Personalised recommendations (month 3)

A BullMQ worker builds a user preference vector from order history and browsing events (stored in a lightweight events table). A nightly job computes top-20 recommended products per user using pgvector ANN search and stores them in a recommendations table. The homepage 'For you' section reads this table.

Phase 3 — AI shopping assistant (month 6)

A chat widget powered by Claude claude-sonnet-4-20250514 (via Anthropic API) with RAG over the product catalogue. The user types a natural language query ('I need a gift under \$50 for my mum who likes gardening'), the system embeds the query, retrieves top-10 matching products via pgvector, and sends them as context to Claude which generates a helpful, personalised recommendation.

Phase 4 — Seller AI tools (month 9)

AI-generated product description writer (seller types a few keywords, AI generates a full description). Automated pricing suggestions based on competitor listings. Demand forecasting using historical order data.

10. CI/CD Pipeline & DevOps

Repository: GitHub monorepo (apps/web, apps/api, packages/shared-types, packages/ui). Branch strategy: main (production), staging (QA), feature/* (development).

- Push to feature/* → GitHub Actions runs lint, type-check, unit tests. PR required to merge.
- Merge to staging → Actions builds Docker image for API, deploys to Railway staging environment. Vercel auto-deploys the Next.js app to a preview URL.
- Merge to main → Actions runs full test suite + integration tests + Playwright E2E. On pass: deploys API to Railway production (rolling deploy, zero downtime). Vercel auto-deploys frontend.
- Database migrations run automatically via Prisma migrate deploy in the API startup script. Migrations are always backward-compatible (no destructive changes without a deprecation period).
- Secrets: stored in GitHub Actions secrets and Railway/Vercel environment variable vaults. Never in code.
- Monitoring: Sentry captures runtime errors. Datadog APM traces every HTTP request and DB query. Alerts fire to Slack for p95 latency > 500ms or error rate > 1%.

11. Development Phases

Phase 1 — Foundation (Weeks 1–4)

- Monorepo setup (Turborepo), ESLint, Prettier, TypeScript strict mode
- Supabase project: schema, RLS policies, Auth configuration
- NestJS API skeleton: AuthModule (Google OAuth, JWT, guest tokens), RBAC guards
- Next.js App Router setup: route groups for admin/seller/buyer
- Tailwind CSS + shadcn/ui component library setup
- CI/CD: GitHub Actions, Vercel project, Railway API service

Phase 2 — Core Commerce (Weeks 5–9)

- ProductModule: CRUD, image upload to Supabase Storage, category management
- CartModule: Redis-backed cart for guests and users, cart merge on login
- OrderModule: checkout pipeline, order state machine, COD flow
- PaymentModule: Stripe PaymentIntent, webhook handler
- Seller portal: product management UI, order dashboard
- Buyer storefront: homepage, category pages, product detail, cart, checkout

Phase 3 — Admin & Notifications (Weeks 10–13)

- Super admin portal: seller management, platform analytics, category admin
- NotificationModule: SendGrid email templates, Twilio SMS, Supabase Realtime order updates
- BullMQ queue setup: email-queue, inventory-queue, export-queue
- Review and rating system
- Payout calculation and history for sellers
- Full audit logging

Phase 4 — AI & Polish (Weeks 14–18)

- pgvector embeddings: product embedding on create/update via queue worker
- Semantic search: replace basic ILIKE search with pgvector cosine similarity
- Performance audit: Lighthouse scores, Core Web Vitals, DB query optimisation
- Security audit: penetration testing checklist, OWASP Top 10 review
- Load testing: k6 scripts simulating 10k concurrent users
- Beta launch with invited sellers and buyers

12. Environment Variables & Configuration

Variable	Purpose
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous key (safe for browser)
SUPABASE_SERVICE_ROLE_KEY	Supabase service role — NestJS only, never browser
DATABASE_URL	PostgreSQL connection string (pooled port 6543)
DIRECT_URL	Direct connection (port 5432) for Prisma migrations
REDIS_URL	Upstash Redis URL
JWT_SECRET	Secret for NestJS JWT signing
STRIPE_SECRET_KEY	Stripe secret key — NestJS only
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Stripe public key for Stripe Elements
SENDGRID_API_KEY	SendGrid transactional email
TWILIO_ACCOUNT_SID	Twilio SMS
TWILIO_AUTH_TOKEN	Twilio SMS
OPENAI_API_KEY	OpenAI embeddings (text-embedding-3-small)
ANTHROPIC_API_KEY	Claude API for AI assistant
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret

13. Folder Structure

```
marketnest/ ■■■ apps/ ■ ■■■ web/ # Next.js 14 (buyer + seller + admin portals) ■ ■
■■■ app/ ■ ■ ■ ■■■ (buyer)/ # Buyer storefront route group ■ ■ ■ ■■■ (seller)/ #
Seller portal route group ■ ■ ■ ■■■ (admin)/ # Super admin route group ■ ■ ■ ■■■
api/ # Next.js API routes (thin, auth only) ■ ■ ■■■ components/ # Shared UI
components ■ ■ ■■■ lib/ # Supabase client, utils ■ ■■■ api/ # NestJS API ■ ■■■
src/ ■ ■■■ auth/ # AuthModule ■ ■■■ products/ # ProductModule ■ ■■■ orders/ #
OrderModule ■ ■■■ payments/ # PaymentModule ■ ■■■ notifications/ #
NotificationModule ■ ■■■ ai/ # AIModule (embeddings, RAG) ■ ■■■ admin/ #
AdminModule ■■■ packages/ ■ ■■■ shared-types/ # TypeScript types shared between
apps ■ ■■■ ui/ # shadcn/ui components (if extracted) ■ ■■■ utils/ # Shared utility
functions ■■■ supabase/ ■ ■■■ migrations/ # SQL migration files ■ ■■■ seed.sql #
Seed data (categories, admin user) ■■■ .github/workflows/ # GitHub Actions CI/CD
■■■ turbo.json # Turborepo config ■■■ package.json # Root workspace
```